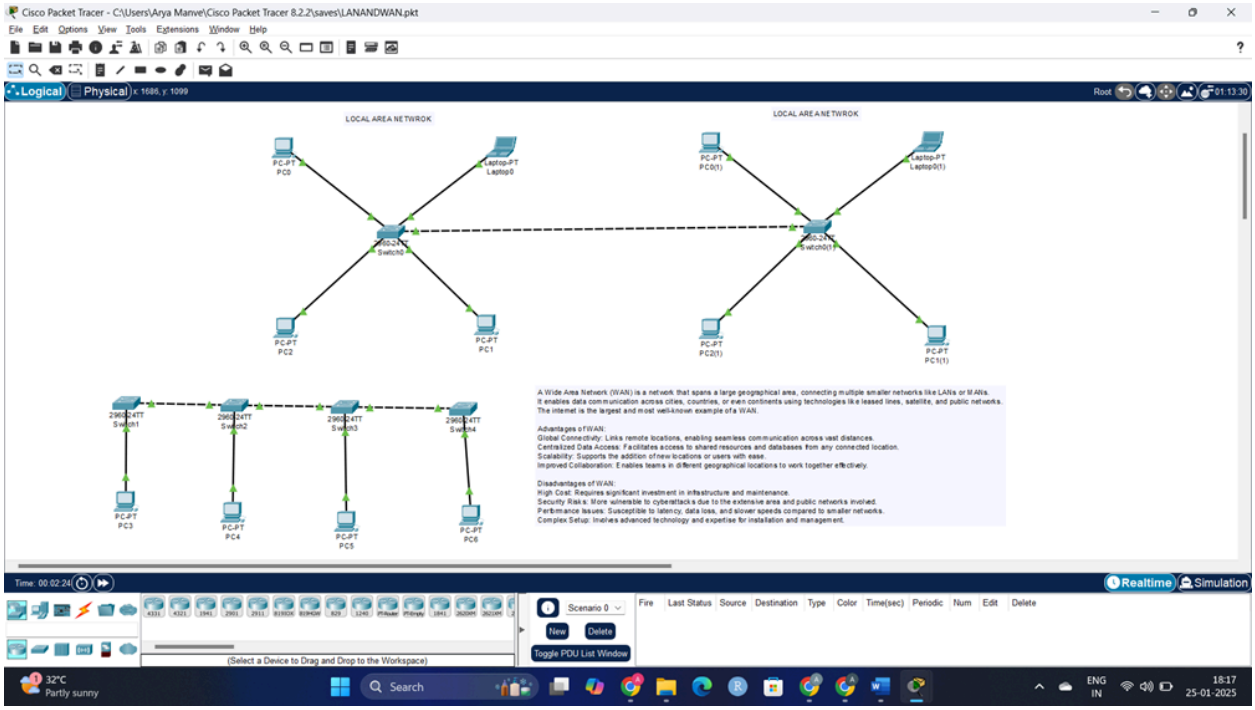
1. **Implement Basic and Advanced Networking Commands( prepare a Report and Upload in PDF format)**

2. **To Establish the LAN,WAN and configuration of the Networking Devices in CISCO Packet Tracer.**
   Wide Area Network (WAN)



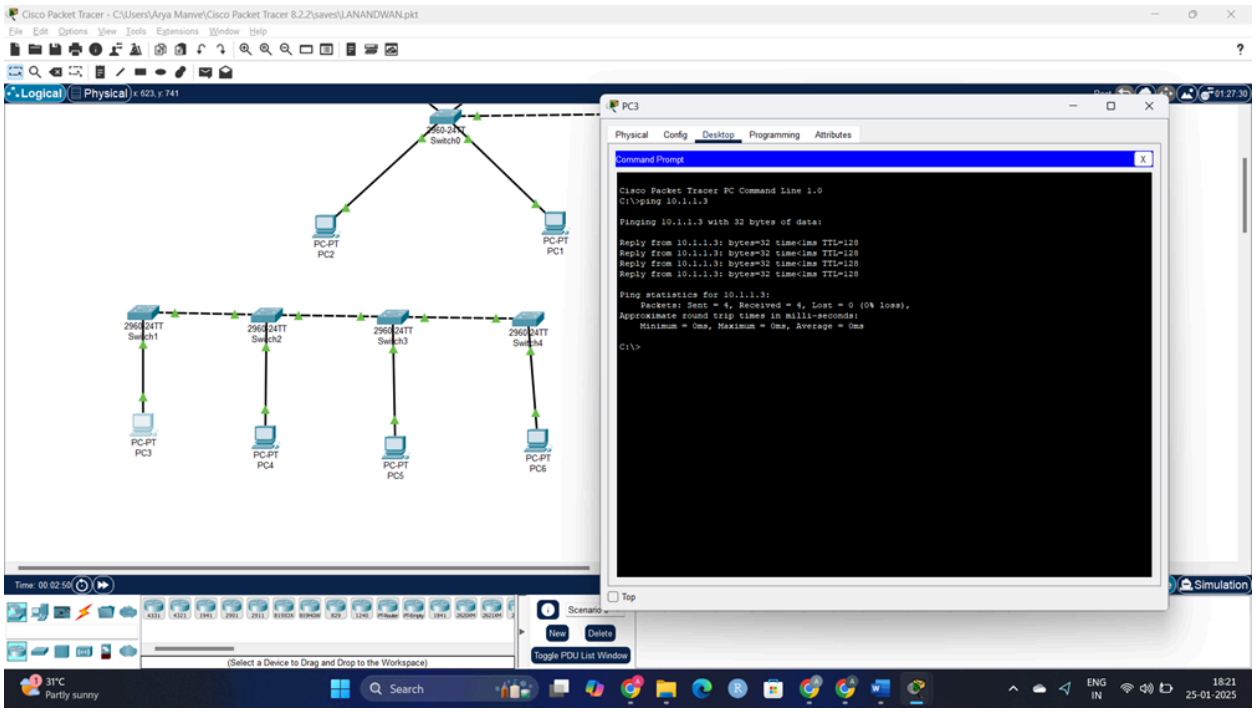

A Wide Area Network (WAN) is a network that spans a large geographical area, connecting multiple smaller networks like LANs or MANs.
It enables data communication across cities, countries, or even continents using technologies like leased lines, satellite, and public networks.
The internet is the largest and most well-known example of a WAN.

Advantages of WAN:
Global Connectivity: Links remote locations, enabling seamless communication across vast distances.
Centralized Data Access: Facilitates access to shared resources and databases from any connected location.
Scalability: Supports the addition of new locations or users with ease.
Improved Collaboration: Enables teams in different geographical locations to work together effectively.

Disadvantages of WAN:
High Cost: Requires significant investment in infrastructure and maintenance.

Security Risks: More vulnerable to cyberattacks due to the extensive area and public networks involved.
Performance Issues: Susceptible to latency, data loss, and slower speeds compared to smaller networks.
Complex Setup: Involves advanced technology and expertise for installation and management.

Ping command from PC_PT PC1 ——> PC-PT PC0(1)



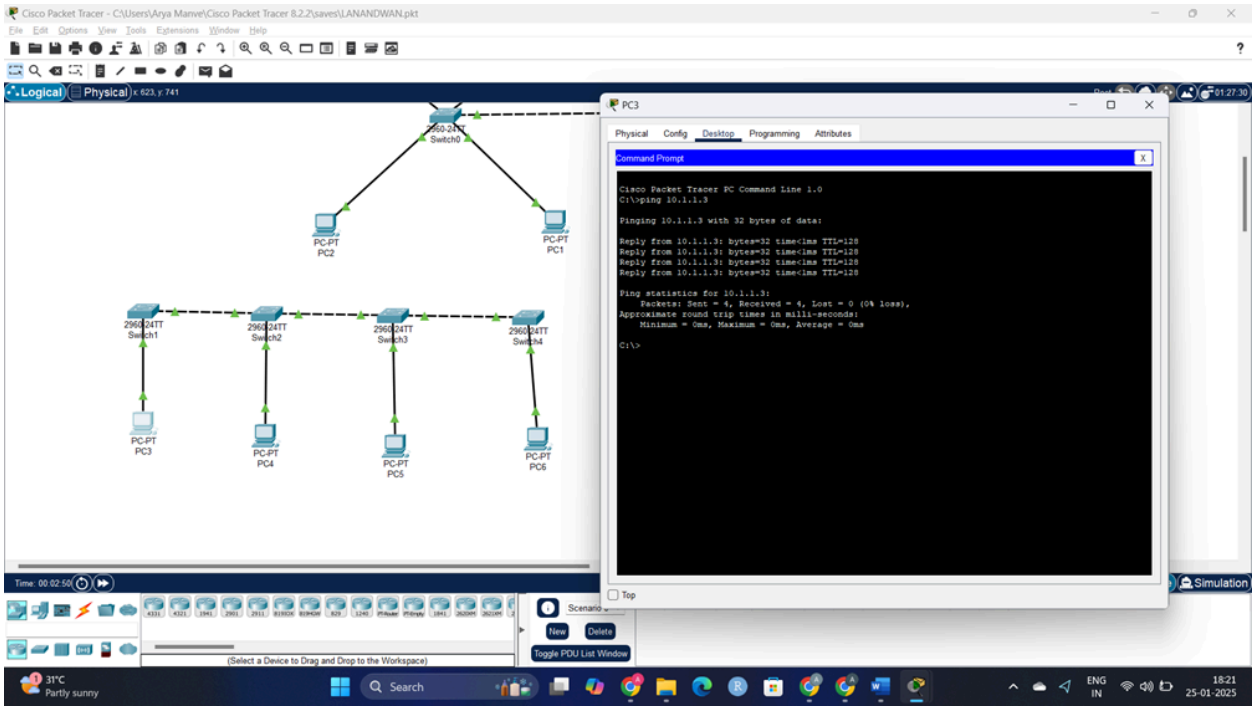

Network Topology

Network topology refers to the arrangement of devices (nodes) and connections (links) within a network. It defines how devices communicate and interact with each other.

Examples:
1. Bus Topology: All devices connect to a single central cable; simple but prone to failure.
2. Star Topology: Devices connect to a central hub; reliable but dependent on the hub.3
3. Ring Topology: Devices connect in a circular fashion; efficient but one failure can disrupt the network.

Bus Topology

Diagram of bus topology network with ping command from PC-PT PC3 —> PC-PT PC6




3. **Write a program to find class and type of a given IP address.**

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
char findClass(int firstOctet) {
 if (firstOctet >= 1 && firstOctet <= 126)
 return 'A';
 else if (firstOctet >= 128 && firstOctet <= 191)
 return 'B';
 else if (firstOctet >= 192 && firstOctet <= 223)
 return 'C';
 else if (firstOctet >= 224 && firstOctet <= 239)
 return 'D';
 else if (firstOctet >= 240 && firstOctet <= 255)
 return 'E';
 else
 return 'X'; // Invalid IP
}
int isPrivate(int firstOctet, int secondOctet) {
 if ((firstOctet == 10) ||
 (firstOctet == 172 && secondOctet >= 16 && secondOctet <= 31) ||
```

```
 (firstOctet == 192 && secondOctet == 168)) {
 return 1;
 }
 return 0;
}
int main() {
 char ip[16];
 int octets[4];
 printf("Enter an IP address (e.g., 192.168.1.1): ");
 scanf("%15s", ip);
 if (sscanf(ip, "%d.%d.%d.%d", &octets[0], &octets[1], &octets[2],
&octets[3]) != 4) {
 printf("Invalid IP address format!\n");
 return 1;
 }
 for (int i = 0; i < 4; i++) {
 if (octets[i] < 0 || octets[i] > 255) {
 printf("Invalid IP address! Each octet must be between 0 and
255.\n");
 return 1;
 }
 }
 char ipClass = findClass(octets[0]);
 if (ipClass == 'X') {
 printf("Invalid IP address class!\n");
 return 1;
 }
 int privateFlag = isPrivate(octets[0], octets[1]);
 printf("IP Address: %s\n", ip);
 printf("Class: %c\n", ipClass);
 printf("Type: %s\n", privateFlag ? "Private" : "Public");
 return 0;
}
```

4. **Write a program for error detection and correction for 7/8 bits ASCII codes using CRC.**

```
#include <stdio.h>
int main() {
```

```
int i, j, k, l;
int fs;
    printf("\n Enter Frame size: ");
    scanf("%d", &fs);
    int f[20];
    printf("\n Enter Frame:");
    for (i = 0; i < fs; i++) {
        scanf("%d", &f[i]);
    }

    int gs;
    printf("\n Enter Generator size: ");
    scanf("%d", &gs);
    int g[20];
    printf("\n Enter Generator:");
    for (i = 0; i < gs; i++) {
        scanf("%d", &g[i]);
    }
    printf("\n Sender Side:");
    printf("\n Frame: ");
    for (i = 0; i < fs; i++) {
        printf("%d", f[i]);
    }
    printf("\n Generator :");
    for (i = 0; i < gs; i++) {
        printf("%d", g[i]);
    }

    int rs = gs - 1;
    printf("\n Number of 0's to be appended: %d", rs);
    for (i = fs; i < fs + rs; i++) {
        f[i] = 0;
    }
    int temp[20];
    for (i = 0; i < 20; i++) {
        temp[i] = f[i];
    }
    printf("\n Message after appending 0's :");
    for (i = 0; i < fs + rs; i++) {
        printf("%d", temp[i]);
```

```
    }

    for (i = 0; i < fs; i++) {
        j = 0;
        k = i;

        if (temp[k] >= g[j]) {
            for (j = 0, k = i; j < gs; j++, k++) {
                if ((temp[k] == 1 && g[j] == 1) || (temp[k] == 0 &&
g[j] ==
0)) {
                    temp[k] = 0;
                } else {
                    temp[k] = 1;
                }
            }
        }
    }

    int crc[15];
    for (i = 0, j = fs; i < rs; i++, j++) {
        crc[i] = temp[j];
    }
    printf("\n CRC bits: ");
    for (i = 0; i < rs; i++) {
        printf("%d", crc[i]);
    }

    printf("\n Transmitted Frame: ");
    int tf[15];
    for (i = 0; i < fs; i++) {
        tf[i] = f[i];
    }
    for (i = fs, j = 0; i < fs + rs; i++, j++) {
        tf[i] = crc[j];
    }
    for (i = 0; i < fs + rs; i++) {
        printf("%d", tf[i]);
    }
    printf("\n Receiver side : ");
```

```
    printf("\n Received Frame: ");
    for (i = 0; i < fs + rs; i++) {
        printf("%d", tf[i]);
    }
    for (i = 0; i < fs + rs; i++) {
        temp[i] = tf[i];
    }

    for (i = 0; i < fs + rs; i++) {
        j = 0;
        k = i;
        if (temp[k] >= g[j]) {
            for (j = 0, k = i; j < gs; j++, k++) {
                if ((temp[k] == 1 && g[j] == 1) || (temp[k] == 0 &&
g[j] ==
0)) {
                    temp[k] = 0;
                } else {
                    temp[k] = 1;
                }
            }
        }
    }
    printf("\n Reaminder: ");
    int rrem[15];
    for (i = fs, j = 0; i < fs + rs; i++, j++) {
        rrem[j] = temp[i];
    }
    for (i = 0; i < rs; i++) {
        printf("%d", rrem[i]);
    }
    int flag = 0;
    for (i = 0; i < rs; i++) {
        if (rrem[i] != 0) {
            flag = 1;
        }
    }
    if (flag == 0) {
        printf("\n Since Remainder Is 0 Hence Message Transmitted
From Sender To Receiver Is Correct");
```

```
    } else {
        printf("\n Since Remainder Is Not 0 Hence Message
Transmitted From Sender To Receiver Contains Error");
    }
    return 0;
}
```

5. **Write a program for error detection and correction for 7/8 bits ASCII codes using HammingCodes.**

```
#include <stdio.h>

int main()
{
    int data[10];
    int dataatrec[10], c, c1, c2, c3, i;

    printf("Enter 4 bits of data one by one\n");
    scanf("%d", &data[7]);
    scanf("%d", &data[6]);
    scanf("%d", &data[5]);
    scanf("%d", &data[3]);

    // Calculation of even parity
    data[4] = data[5] ^ data[6] ^ data[7];
    data[2] = data[3] ^ data[6] ^ data[7];
    data[1] = data[3] ^ data[5] ^ data[7];

    printf("\nEncoded data is\n");
    for (i = 1; i <= 7; i++)
        printf("%d", data[i]);

    printf("\n\nEnter received data bits one by one\n");
    for (i = 1; i <= 7; i++)
        scanf("%d", &dataatrec[i]);
```

```
    c1 = dataatrec[1] ^ dataatrec[3] ^ dataatrec[5] ^ dataatrec[7];
    c2 = dataatrec[2] ^ dataatrec[3] ^ dataatrec[6] ^ dataatrec[7];
    c3 = dataatrec[4] ^ dataatrec[5] ^ dataatrec[6] ^ dataatrec[7];

    c = c3 * 4 + c2 * 2 + c1;

    if (c == 0)
    {
        printf("\nCongratulations, there is no error.\n");
    }
    else
    {
        printf("\nError at position: %d\n", c);
        printf("Corrected message is: ");
        dataatrec[c] = dataatrec[c] == 0 ? 1 : 0;
        for (i = 1; i <= 7; i++)
            printf("%d", dataatrec[i]);
        printf("\n");
    }

    return 0;
}
```

6. Write a program to simulate
   i) Go back N  Sliding Window Protocol in peer to peer mode
   ii) Selective Repeat Sliding Window Protocol in peer to peer mode

   Different doc

7. Implement Socket programming to create Client and Server to send Hello message. (Implement TCP and UDP Socket)

sevrer.py

```
import socket

def tcp_server():
    host = 'localhost'
```

```python
    port = 12345
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.bind((host, port))
        s.listen(1)
        print("TCP Server listening...")
        conn, addr = s.accept()
        with conn:
            print(f"Connected by {addr}")
            data = conn.recv(1024).decode()
            print("Received (TCP):", data)
            conn.sendall("Hello received".encode())

def udp_server():
    host = 'localhost'
    port = 12345
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.bind((host, port))
        print("UDP Server listening...")
        data, addr = s.recvfrom(1024)
        print("Received (UDP):", data.decode())
        s.sendto("Hello received".encode(), addr)

def menu():
    while True:
        print("\n--- Server Menu ---")
        print("1. Start TCP Server")
        print("2. Start UDP Server")
        print("3. Exit")
        choice = input("Enter choice: ")
        if choice == '1':
            tcp_server()
        elif choice == '2':
            udp_server()
        elif choice == '3':
            break
        else:
            print("Invalid choice!")

if __name__ == '__main__':
    menu()
```

client.py

```python
import socket

def tcp_client():
    host = 'localhost'
    port = 12345
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((host, port))
        s.sendall("Hello".encode())
        data = s.recv(1024)
        print("Received (TCP):", data.decode())

def udp_client():
    host = 'localhost'
    port = 12345
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.sendto("Hello".encode(), (host, port))
        data, _ = s.recvfrom(1024)
        print("Received (UDP):", data.decode())

def menu():
    while True:
        print("\n--- Client Menu ---")
        print("1. Send TCP Hello")
        print("2. Send UDP Hello")
        print("3. Exit")
        choice = input("Enter choice: ")
        if choice == '1':
            tcp_client()
        elif choice == '2':
            udp_client()
        elif choice == '3':
            break
        else:
            print("Invalid choice!")

if __name__ == '__main__':
    menu()
```

8. Write a program to demonstrate subnetting and find the subnet masks.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

void calculateSubnetMask(int subnetBits, int subnetMask[]) {
    for (int i = 0; i < 4; i++) {
        if (subnetBits >= 8) {
            subnetMask[i] = 255;
            subnetBits -= 8;
        } else if (subnetBits > 0) {
            subnetMask[i] = 256 - (1 << (8 - subnetBits));
            subnetBits = 0;
        } else {
            subnetMask[i] = 0;
        }
    }
}

void printIP(int ip[]) {
    printf("%d.%d.%d.%d\n", ip[0], ip[1], ip[2], ip[3]);
}

char getIPClass(int firstOctet) {
    if (firstOctet >= 1 && firstOctet <= 126) return 'A';
    if (firstOctet >= 128 && firstOctet <= 191) return 'B';
    if (firstOctet >= 192 && firstOctet <= 223) return 'C';
    return 'X';
}

void getDefaultSubnetMask(char ipClass, int subnetMask[]) {
    switch (ipClass) {
        case 'A': subnetMask[0] = 255; subnetMask[1] = 0;   subnetMask[2]
= 0;   subnetMask[3] = 0;   break;
```

```
        case 'B': subnetMask[0] = 255; subnetMask[1] = 255; subnetMask[2]
= 0;   subnetMask[3] = 0;   break;
        case 'C': subnetMask[0] = 255; subnetMask[1] = 255; subnetMask[2]
= 255; subnetMask[3] = 0;   break;
        default:  subnetMask[0] = subnetMask[1] = subnetMask[2] =
subnetMask[3] = 0;
    }
}

int validateIP(char *ipAddress, int ip[]) {
    return sscanf(ipAddress, "%d.%d.%d.%d", &ip[0], &ip[1], &ip[2],
&ip[3]) == 4 &&
           ip[0] >= 0 && ip[0] <= 255 && ip[1] >= 0 && ip[1] <= 255 &&
           ip[2] >= 0 && ip[2] <= 255 && ip[3] >= 0 && ip[3] <= 255;
}

int main() {
    char ipAddress[16];
    int subnetBits;
    int ip[4] = {0};
    int subnetMask[4] = {0};

    printf("Enter IP address: ");
    scanf("%15s", ipAddress);

    if (!validateIP(ipAddress, ip)) {
        printf("Invalid IP address!\n");
        return 1;
    }

    char ipClass = getIPClass(ip[0]);
    if (ipClass == 'X') {
        printf("Invalid or unsupported IP class!\n");
        return 1;
    }

    getDefaultSubnetMask(ipClass, subnetMask);
    printf("\nIP Class: %c\n", ipClass);
    printf("Default Subnet Mask: ");
    printIP(subnetMask);
```

```
    printf("Enter the number of subnet bits: ");
    if (scanf("%d", &subnetBits) != 1 || subnetBits < 0 || subnetBits >
32) {
        printf("Invalid subnet bits! Must be between 0 and 32.\n");
        return 1;
    }

    calculateSubnetMask(subnetBits, subnetMask);

    printf("\nIP Address: ");
    printIP(ip);
    printf("Calculated Subnet Mask: ");
    printIP(subnetMask);

    return 0;
}
```

9. Write a program using TCP socket for wired network for following:

a. Say Hello to Each other b. File transfer c. Calculator (Arithmetic)

server.py

```
import socket
import threading

# Server configurations
HOST = '127.0.0.1'
PORT = 1234

# Handle client communication
def handle_client(client_socket):
    try:
        # Send greeting
        client_socket.send(b"Server: Hello!")
        print("Server: Hello sent")

        # File transfer
        file_name = "server_file.txt"
```

```
        # Create a sample file
        with open(file_name, "wb") as f:
            f.write(b"This is a sample file content.")

        # Send the file
        with open(file_name, "rb") as f:
            data = f.read(1024)
            while data:
                client_socket.send(data)
                data = f.read(1024)

        # Send marker to indicate the end of file transfer
        client_socket.send(b"<END_FILE>")
        print("File sent")

        # Calculator functionality
        while True:
            expression = client_socket.recv(1024).decode('utf-8')
            if expression.lower() == "exit":
                print("Client disconnected.")
                break
            try:
                # Using eval safely (for demonstration purposes, better
alternatives exist)
                result = str(eval(expression))
            except Exception as e:
                result = f"Error: {str(e)}"

            client_socket.send(result.encode('utf-8'))

    except Exception as e:
        print(f"Error during client handling: {e}")
    finally:
        client_socket.close()

# Start server
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind((HOST, PORT))
server_socket.listen(5)
```

```
print("Server started. Waiting for clients...")

while True:
    client, addr = server_socket.accept()
    print(f"Client connected: {addr}")
    thread = threading.Thread(target=handle_client, args=(client,))
    thread.start()
```

Client.py

```
import socket

# Server configurations
SERVER_IP = '127.0.0.1'
SERVER_PORT = 1234

# Connect to the server
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect((SERVER_IP, SERVER_PORT))

# Receive greeting
greeting = client_socket.recv(1024).decode('utf-8')
print(f"Server: {greeting}")

# File reception
file_name = "received_file.txt"
with open(file_name, "wb") as f:
    while True:
        data = client_socket.recv(1024)
        if b"<END_FILE>" in data:
            # Remove the marker before writing the file
            f.write(data.replace(b"<END_FILE>", b""))
            break
        f.write(data)

print("File received")

# Calculator functionality
print("\nCalculator (Type 'exit' to quit)")
while True:
```

```
    expression = input("Enter arithmetic expression (or 'exit' to quit): ")
    client_socket.send(expression.encode('utf-8'))
    if expression.lower() == "exit":
        break
    result = client_socket.recv(1024).decode('utf-8')
    print(f"Result: {result}")

# Close connection
client_socket.close()
```

10. Write a program using UDP Sockets to enable file transfer (Script, Text, Audio and Video one file each) between two machines.

Udp_server.py

```
import socket
import os

SERVER_IP = "0.0.0.0"
SERVER_PORT = 9999
BUFFER_SIZE = 1024

# Base folders for each file type
file_paths = {
    "script": "./files/scripts/",
    "text": "./files/texts/",
    "audio": "./files/audio/",
    "video": "./files/video/"
}

def start_server():
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server_socket.bind((SERVER_IP, SERVER_PORT))
    print(f"Server listening on {SERVER_IP}:{SERVER_PORT}")

    while True:
```

```
        print("\n[Server Menu]")
        print("Waiting for client request...")

        data, client_address = server_socket.recvfrom(BUFFER_SIZE)
        request = data.decode().split('|')
        if len(request) != 2:
            server_socket.sendto("Invalid request format.".encode(),
client_address)
            continue

        file_type, file_name = request
        base_path = file_paths.get(file_type.lower())

        if not base_path:
            server_socket.sendto("Invalid file type.".encode(),
client_address)
            continue

        full_path = os.path.join(base_path, file_name)

        if not os.path.isfile(full_path):
            server_socket.sendto("File not found.".encode(),
client_address)
            continue

        server_socket.sendto("FOUND".encode(), client_address)

        with open(full_path, 'rb') as f:
            while True:
                chunk = f.read(BUFFER_SIZE)
                if not chunk:
                    break
                server_socket.sendto(chunk, client_address)

        # Send a special message to denote end of file
        server_socket.sendto("EOF".encode(), client_address)
        print(f"Sent file '{file_name}' of type '{file_type}' to
{client_address}")

if __name__ == "__main__":
```

```
    start_server()
```

Udp_client.py

```
import socket

SERVER_IP = "127.0.0.1"  # Change to server's IP if remote
SERVER_PORT = 9999
BUFFER_SIZE = 1024

def show_menu():
    print("\n[Client Menu]")
    print("1. Script File")
    print("2. Text File")
    print("3. Audio File")
    print("4. Video File")
    print("5. Exit")

def get_file_type(choice):
    return {
        "1": "script",
        "2": "text",
        "3": "audio",
        "4": "video"
    }.get(choice, None)

def request_file():
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    show_menu()
    choice = input("Enter your choice: ")
    file_type = get_file_type(choice)

    if not file_type:
        print("Invalid choice. Exiting...")
        return

    file_name = input(f"Enter the name of the {file_type} file (with
extension): ")
```

```
    request = f"{file_type}|{file_name}"
    client_socket.sendto(request.encode(), (SERVER_IP, SERVER_PORT))

    response, _ = client_socket.recvfrom(BUFFER_SIZE)
    if response.decode() != "FOUND":
        print("Server response:", response.decode())
        return

    with open("client_" + file_name, 'wb') as f:
        while True:
            data, _ = client_socket.recvfrom(BUFFER_SIZE)
            if data.decode(errors='ignore') == "EOF":
                break
            f.write(data)

    print(f"File '{file_name}' received successfully and saved as
'client_{file_name}'")

if __name__ == "__main__":
    while True:
        request_file()
        cont = input("Do you want to download another file? (y/n): ")
        if cont.lower() != 'y':
            print("Exiting client.")
            break
```

11. Write a program to implement: a. Network Routing: Shortest path routing

```
#include <stdio.h>
#include <limits.h>

#define MAX 100
#define INF 99999

int findMinDistance(int dist[], int visited[], int n) {
    int min = INF, min_index = -1;
    for (int v = 0; v < n; v++) {
        if (!visited[v] && dist[v] <= min) {
```

```
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
}

void printPath(int parent[], int j) {
    if (parent[j] == -1)
        return;

    printPath(parent, parent[j]);
    printf(" -> %d", j);
}

void printSolution(int dist[], int parent[], int src, int n) {
    printf("\nRouting Table (From Node %d):\n", src);
    printf("Destination\tDistance\tPath\n");

    for (int i = 0; i < n; i++) {
        if (i != src) {
            printf("%d\t\t%d\t\t%d", i, dist[i], src);
            printPath(parent, i);
            printf("\n");
        }
    }
}

void dijkstra(int graph[MAX][MAX], int src, int n) {
    int dist[MAX];       // Shortest distance from src to i
    int visited[MAX];    // True if node is processed
    int parent[MAX];     // To store the path

    for (int i = 0; i < n; i++) {
        dist[i] = INF;
        visited[i] = 0;
        parent[i] = -1;
    }

    dist[src] = 0;
```

```
    for (int count = 0; count < n - 1; count++) {
        int u = findMinDistance(dist, visited, n);
        visited[u] = 1;

        for (int v = 0; v < n; v++) {
            if (!visited[v] && graph[u][v] && dist[u] != INF
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
                parent[v] = u;
            }
        }
    }

    printSolution(dist, parent, src, n);
}

int main() {
    int n;
    int graph[MAX][MAX];

    printf("Enter number of nodes in the network: ");
    scanf("%d", &n);

    printf("Enter the cost adjacency matrix (use 0 for no direct
edge):\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
            if (graph[i][j] == 0 && i != j)
                graph[i][j] = INF;
        }
    }

    int src;
    printf("Enter the source node (0 to %d): ", n - 1);
    scanf("%d", &src);

    dijkstra(graph, src, n);
```

```
    return 0;
}
```